OOPJ Solved Question Bank

Object Oriented Programming with Java (C2311C1)

Detailed edition: concept, step-by-step explanation, dry run, output and exam notes for every question

Part	Contents	Marks
A	Predict the Output: 5 programs, each with a line-by-line trace	Quick marks
B	Long Answer / Programming: 15 questions, each with concept, code, trace, output, viva points	5 marks each
C	Theory: 18 questions, each with definition, explanation, example, diagram/table	3 marks each
D	Exam strategy, answer-writing format, MCQ revision	Q1 MCQs

Paper pattern: Q1 MCQs | Q2 3-mark questions (attempt any 3 of 5) | Q3 5-mark questions (attempt any 3 of 5).

How to use this PDF

- Every Java program in this PDF was **compiled and executed**, and the output shown is the real output.
- For a **5-mark** answer write: definition or concept (1), code (2.5), output (1), one-line explanation (0.5).
- For a **3-mark** answer write: definition (1), explanation or points (1), example or table (1).
- Yellow boxes are **exam notes**: extra points that earn marks or avoid common mistakes.

PART A: Predict the Output

Method for every program: (1) find the entry point `main`, (2) go statement by statement, (3) when an object is created, trace the constructor chain, (4) write the output exactly, line by line.

Program 1: Constructor chaining in multilevel inheritance

```
class A { A() { System.out.println("A"); } }
class B extends A { B() { System.out.println("B"); } }
class C extends B { C() { System.out.println("C"); } }
public class Main {
    public static void main(String[] args) {
        new C();
    }
}
```

Output:

```
A
B
C
```

Step-by-step trace

Step	What happens
1	`new C()` starts. The first line of any constructor is a call to the parent constructor. C() has none written, so the compiler inserts `super();`.
2	Control goes to `B()`. It also begins with an implicit `super();`, so control goes to `A()`.
3	`A()` has no parent except `Object`. It runs its body: prints **A**.
4	A finishes, control returns to `B()`. It runs its body: prints **B**.
5	B finishes, control returns to `C()`. It runs its body: prints **C**.

Rule to remember: constructors are *called* bottom-up (C, B, A) but *finish and print* top-down (A, B, C). The parent part of an object must be fully built before the child part.

Program 2: Runtime polymorphism

```
class Animal { void sound() { System.out.println("Animal sound"); } }
class Dog extends Animal { void sound() { System.out.println("Bark"); } }
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
        a = new Animal();
        a.sound();
    }
}
```

Output:

```
Bark
Animal sound
```

Step-by-step trace

Statement	Reference type	Actual object	Method that runs
`Animal a = new Dog();`	Animal	Dog	-
`a.sound();`	Animal	Dog	Dog's `sound()` prints **Bark**
`a = new Animal();`	Animal	Animal	-
`a.sound();`	Animal	Animal	Animal's `sound()` prints **Animal sound**

An overridden method is chosen using the **actual object** at runtime (dynamic method dispatch), not the reference type.

Program 3: Static counter and instance id

```
class Counter {
    static int count = 0;
    int id;
    Counter() { count++; id = count; }
}
public class Main {
    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        System.out.println(c1.id + " " + c2.id + " " + Counter.count);
    }
}
```

Output:

```
1 2 2
```

Step-by-step trace

After statement	count (static, shared)	c1.id	c2.id
initial	0	-	-
`new Counter()` for c1: count++ then id = count	1	1	-
`new Counter()` for c2: count++ then id = count	2	1	2

The print statement joins `c1.id` (1), a space, `c2.id` (2), a space, `Counter.count` (2). Result: `1 2 2`. `id` is an instance variable (each object has its own copy, so c1 keeps 1), while `count` is static (one copy shared by all).

Program 4: Recursion (factorial of 5)

```
public class Main {
    static int f(int n) {
        if (n <= 1) return 1;
        return n * f(n - 1);
    }
    public static void main(String[] args) {
        System.out.println(f(5));
    }
}
```

Output:

```
120
```

Step-by-step trace

```
Going down (calls)            Coming up (returns)
f(5) = 5 * f(4)               f(1) returns 1
 f(4) = 4 * f(3)              f(2) returns 2 * 1  = 2
  f(3) = 3 * f(2)             f(3) returns 3 * 2  = 6
   f(2) = 2 * f(1)            f(4) returns 4 * 6  = 24
    f(1) -> base case, 1      f(5) returns 5 * 24 = 120
```

Five calls are stacked; the base case `n <= 1` stops the recursion; the results are multiplied while the stack unwinds. Output: `120`.

Program 5: Constructor chaining with this()

```
class P {
    int x, y;
    P() {
        this(5, 10);
        System.out.println("Default");
    }
    P(int x, int y) {
        this.x = x;
        this.y = y;
        System.out.println("Param: " + x + "," + y);
    }
}
public class Main {
    public static void main(String[] args) {
        new P();
    }
}
```

Output:

```
Param: 5,10
Default
```

Step-by-step trace

Step	What happens
1	`new P()` calls the no-argument constructor `P()`.
2	Its first statement is `this(5, 10)`, which jumps to `P(int x, int y)` **before** anything else in `P()` runs.
3	`P(5,10)` sets x=5, y=10 and prints **Param: 5,10**.
4	Control returns to `P()`, which continues with the next statement and prints **Default**.

The statement written after `this(...)` runs **last**, so the output order is the reverse of how the constructors appear in the code.

PART B: Long Answer / Programming Questions (5 marks)

Q1. Dynamic method dispatch + Shape program

Concept

Dynamic method dispatch is the mechanism by which a call to an **overridden** method is resolved at **runtime** instead of compile time. When a superclass reference variable holds the address of a subclass object, the version of the method that runs is decided by the **type of the object** (not the type of the reference). This is how Java achieves **runtime polymorphism**.

Requirements for it to work

- There must be inheritance (IS-A relationship): `Circle` is-a `Shape`.
- The subclass must **override** the superclass method (same name, same parameters).
- A **superclass reference** must point to a **subclass object**: `Shape s = new Circle();` (upcasting).

Program

```
class Shape {
    void area() { System.out.println("Area of generic shape"); }
}
class Circle extends Shape {
    double r = 5;
    @Override
    void area() { System.out.println("Area of Circle = " + (3.14 * r * r)); }
}
class Rectangle extends Shape {
    double l = 4, b = 6;
    @Override
    void area() { System.out.println("Area of Rectangle = " + (l * b)); }
}
public class Main {
    public static void main(String[] args) {
        Shape s;              // superclass reference
        s = new Circle();     // points to a Circle object
        s.area();             // Circle's area() runs
        s = new Rectangle();  // same reference, now points to a Rectangle object
        s.area();             // Rectangle's area() runs
    }
}
```

Output:

```
Area of Circle = 78.5
Area of Rectangle = 24.0
```

Explanation

- `Shape s;` creates only a reference of type Shape. No object exists yet.
- `s = new Circle();` is upcasting. `s` now holds the address of a Circle object.
- `s.area()`: the compiler only checks that `Shape` has an `area()` method. At runtime the JVM looks at the actual object (Circle) and runs `Circle.area()`.
- `s = new Rectangle();` the same reference now points to a Rectangle, so the same call `s.area()` runs `Rectangle.area()`.
- Calculation: Circle = 3.14 x 5 x 5 = 78.5. Rectangle = 4 x 6 = 24.0 (double, so it prints `24.0`).

Write the words **runtime polymorphism**, **upcasting** and **method overriding** in your answer. Also note: a superclass reference can call only methods that exist in the superclass; the subclass-specific methods are not accessible through it.

Q2. Multilevel inheritance with the super keyword

Concept

In **multilevel inheritance** a class is derived from a class that is itself derived from another class: Person, Employee, Manager. When a Manager object is created, constructors run from the top of the chain: Person, then Employee, then Manager. Since Person and Employee have **parameterized** constructors, each child must pass values upward using `super(...)`, which must be the **first statement** in the constructor.

```
Person   (name)
   ^
   | extends
Employee (empId)
   ^
   | extends
Manager  (department)
```

Program

```
class Person {
    String name;
    Person(String name) {
        this.name = name;
    }
}
class Employee extends Person {
    int empId;
    Employee(String name, int empId) {
        super(name);              // calls Person(String)
        this.empId = empId;
    }
}
class Manager extends Employee {
    String department;
    Manager(String name, int empId, String department) {
        super(name, empId);       // calls Employee(String, int)
        this.department = department;
    }
    void display() {
        System.out.println("Name       : " + name);
        System.out.println("Employee ID: " + empId);
        System.out.println("Department : " + department);
    }
}
public class Main {
    public static void main(String[] args) {
        Manager m = new Manager("Rahul", 101, "Finance");
        m.display();
    }
}
```

Output:

```
Name       : Rahul
Employee ID: 101
Department : Finance
```

Execution trace for `new Manager("Rahul", 101, "Finance")`

Order	Constructor	Action
1	Manager(...) starts	First line `super(name, empId)` calls Employee
2	Employee(...) starts	First line `super(name)` calls Person
3	Person(...) runs	Sets name = Rahul. Finishes.
4	Employee continues	Sets empId = 101. Finishes.
5	Manager continues	Sets department = Finance. Finishes.

`display()` uses `name` and `empId` directly because Manager **inherits** them from Employee and Person (they are not private). If Person had no no-arg constructor and Employee did not call `super(name)`, the program would not compile.

Q3. Interface Vehicle with Car and Bike, plus instanceof

Concept

An **interface** declares **what** a class must do, not **how**. Classes use `implements` to promise to provide all of its methods. An interface reference can point to any object of an implementing class, which gives polymorphism. The `instanceof` operator checks the real type of the object at runtime.

Program

```
interface Vehicle {
    void start();
    void stop();
}
class Car implements Vehicle {
    public void start() { System.out.println("Car started"); }
    public void stop()  { System.out.println("Car stopped"); }
}
class Bike implements Vehicle {
    public void start() { System.out.println("Bike started"); }
    public void stop()  { System.out.println("Bike stopped"); }
}
public class Main {
    public static void main(String[] args) {
        Vehicle v1 = new Car();     // interface reference -> Car object
        Vehicle v2 = new Bike();    // interface reference -> Bike object

        v1.start();
        v1.stop();
        v2.start();
        v2.stop();

        if (v1 instanceof Car)  System.out.println("v1 is a Car");
        if (v1 instanceof Bike) System.out.println("v1 is a Bike");
        if (v2 instanceof Car)  System.out.println("v2 is a Car");
        if (v2 instanceof Bike) System.out.println("v2 is a Bike");
    }
}
```

Output:

```
Car started
Car stopped
Bike started
Bike stopped
v1 is a Car
v2 is a Bike
```

Explanation

- `Vehicle v1 = new Car();` : you cannot create an object of an interface, but you can create a **reference** of it.
- `v1.start()` runs `Car.start()` (dynamic dispatch). `v2.start()` runs `Bike.start()`.
- `v1 instanceof Car` is true; `v1 instanceof Bike` is false because the real object is a Car. Hence only two of the four `if` lines print.

Common mistake: writing `void start() {...}` in Car without `public`. Interface methods are implicitly `public abstract`, and an overriding method cannot have weaker access, so it gives a compile error: *attempting to assign weaker access privileges*.

Q4. Interface inheritance (A, B extends A, class implements B)

Concept

An interface can extend another interface using `extends` (and even several, separated by commas). The child interface inherits all abstract methods of the parent. A class that implements the child interface must therefore define **all** methods: the parent's and the child's.

```
interface A  { showA() }
     ^
     | extends
interface B  { showB() }   (B also has showA() by inheritance)
     ^
     | implements
class Demo   -> must define showA() and showB()
```

Program

```
interface A {
    void showA();
}
interface B extends A {
    void showB();
}
class Demo implements B {
    public void showA() { System.out.println("Method showA() from interface A"); }
    public void showB() { System.out.println("Method showB() from interface B"); }
}
public class Main {
    public static void main(String[] args) {
        Demo d = new Demo();
        d.showA();
        d.showB();
    }
}
```

Output:

```
Method showA() from interface A
Method showB() from interface B
```

Why the class must define both methods

`B extends A` means B contains `showA()` (inherited) and `showB()` (own). A concrete class that implements B accepts the whole contract. If `Demo` defined only `showB()`, the compiler would report: *Demo is not abstract and does not override abstract method showA() in A*. The only way out is to declare `Demo` as `abstract`.

Keyword memory: **class extends class**, **class implements interface**, **interface extends interface**.

Q5. Rectangle class with constructor chaining using this()

Concept

Constructor overloading gives a class several constructors with different parameter lists. **Constructor chaining** with `this(...)` lets one constructor call another constructor of the same class so the initialization code is written only once. `this(...)` must be the **first statement**.

Program

```
class Rectangle {
    int length, breadth;

    Rectangle() {                        // no-argument constructor
        this(1, 1);                      // chains to the parameterized constructor
    }
    Rectangle(int length, int breadth) { // parameterized constructor
        this.length = length;
        this.breadth = breadth;
    }
    int area() {
        return length * breadth;
    }
}
public class Main {
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle();        // length = 1, breadth = 1
        Rectangle r2 = new Rectangle(5, 4);    // length = 5, breadth = 4
        System.out.println("Area of r1 = " + r1.area());
        System.out.println("Area of r2 = " + r2.area());
    }
}
```

Output:

```
Area of r1 = 1
Area of r2 = 20
```

Explanation

- `new Rectangle()` calls the no-arg constructor, which immediately executes `this(1, 1)`. That hands control to the two-argument constructor, which stores length = 1 and breadth = 1.
- `new Rectangle(5, 4)` calls the two-argument constructor directly.
- `this.length = length;` : `this.length` is the instance variable, `length` is the parameter. `this` removes the name clash.
- Area r1 = 1 x 1 = 1, area r2 = 5 x 4 = 20.

Never use `this()` in a loop of constructors calling each other (A calls B and B calls A): that is a compile error (*recursive constructor invocation*).

Q6. Method overloading: area() of three shapes

Concept

Method overloading means several methods in the same class share the same name but differ in the **number, type, or order of parameters**. The compiler picks the correct version from the arguments at **compile time**, so it is called compile-time (static) polymorphism. Return type alone cannot distinguish overloaded methods.

Program

```
class Area {
    int area(int side) {                // square: one int
        return side * side;
    }
    int area(int length, int breadth) { // rectangle: two ints
        return length * breadth;
    }
    double area(double radius) {        // circle: one double
        return 3.14 * radius * radius;
    }
}
public class Main {
    public static void main(String[] args) {
        Area a = new Area();
        System.out.println("Square    : " + a.area(5));
        System.out.println("Rectangle : " + a.area(4, 6));
        System.out.println("Circle    : " + a.area(2.0));
    }
}
```

Output:

```
Square    : 25
Rectangle : 24
Circle    : 12.56
```

How the compiler chooses

Call	Arguments	Version chosen	Result
`a.area(5)`	one int	`area(int)`	5 x 5 = 25
`a.area(4, 6)`	two ints	`area(int, int)`	4 x 6 = 24
`a.area(2.0)`	one double	`area(double)`	3.14 x 2.0 x 2.0 = 12.56

`a.area(2)` (int literal) would call the **square** version and return 4, not the circle. A literal like `2.0` is a double.

Q7. Recursion: factorial, Fibonacci, and the call stack

Concept

Recursion is when a method calls itself. Every recursive method needs a **base case** (stops the recursion) and a **recursive case** (reduces the problem towards the base case). Each call is stored as a **stack frame** on the call stack.

Program

```
public class Main {
    // factorial: n! = n * (n-1)!   base case: 0! = 1! = 1
    static int factorial(int n) {
        if (n <= 1) return 1;
        return n * factorial(n - 1);
    }
    // n-th Fibonacci number: fib(n) = fib(n-1) + fib(n-2); base: fib(0)=0, fib(1)=1
    static int fib(int n) {
        if (n <= 1) return n;
        return fib(n - 1) + fib(n - 2);
    }
    // prints the first 'terms' Fibonacci numbers
    static void printFibonacci(int terms) {
        for (int i = 0; i < terms; i++) {
            System.out.print(fib(i) + " ");
        }
        System.out.println();
    }
    public static void main(String[] args) {
        System.out.println("Factorial of 4 = " + factorial(4));
        System.out.print("Fibonacci series (7 terms): ");
        printFibonacci(7);
    }
}
```

Output:

```
Factorial of 4 = 24
Fibonacci series (7 terms): 0 1 1 2 3 5 8
```

How the call stack unwinds for factorial(4)

```
PHASE 1: calls are PUSHED (stack grows)       PHASE 2: results RETURN (stack unwinds)

| factorial(1) |  <- top, base case: returns 1
| factorial(2) |  waits for factorial(1)        factorial(1) returns 1
| factorial(3) |  waits for factorial(2)        factorial(2) returns 2 * 1 = 2
| factorial(4) |  waits for factorial(3)        factorial(3) returns 3 * 2 = 6
| main()       |  bottom                        factorial(4) returns 4 * 6 = 24
```

- **Push phase:** `factorial(4)` cannot finish until it knows `factorial(3)`, so it waits on the stack. This continues until `factorial(1)`.
- **Base case:** `factorial(1)` returns 1 immediately without another call.
- **Pop phase:** frames are removed in reverse (last in, first out). Each frame multiplies its own `n` by the value it receives and returns upward.
- Final answer: 4 x (3 x (2 x 1)) = 24.

Fibonacci terms

i	0	1	2	3	4	5	6
fib(i)	0	1	1	2	3	5	8

Without a base case the recursion never stops and the program crashes with `StackOverflowError`. Recursive Fibonacci recomputes values repeatedly (slow for large n); a loop is faster, but the question asks for recursion.

Q8. Static variable counting objects

Concept

A **static variable** belongs to the **class**: one copy shared by all objects, created when the class is loaded. An **instance variable** has a separate copy in every object. A **static method** can be called with the class name and can access only static members directly.

Program

```
class Counter {
    static int count = 0;                // shared by all objects

    Counter() {
        count++;                         // runs once per object created
    }
    static void displayCount() {
        System.out.println("Objects created: " + count);
    }
}
public class Main {
    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter.displayCount();
        Counter c2 = new Counter();
        Counter.displayCount();
        Counter c3 = new Counter();
        Counter.displayCount();
    }
}
```

Output:

```
Objects created: 1
Objects created: 2
Objects created: 3
```

Why the variable must be static

- Static means **one common copy**. Every constructor call increments the same variable, so it remembers how many objects exist.
- If `count` were an instance variable, each new object would get its own `count` starting at 0, become 1 in its constructor, and the program would always print 1.
- The method `displayCount()` is static so it can be called as `Counter.displayCount()` without needing any object, and it can legally read the static `count`.

After	Static count
c1 created	1
c2 created	2
c3 created	3

Q9. Passing and returning objects: Complex numbers

Concept

In Java, an object can be passed to a method as an argument, and a method can return an object. What is passed is a **copy of the reference**, so both the caller and the method refer to the **same object**. Returning an object lets a method hand back a newly created result.

Program

```
class Complex {
    int real, imag;

    Complex(int real, int imag) {
        this.real = real;
        this.imag = imag;
    }
    // receives a Complex object and returns a new Complex object
    Complex add(Complex c) {
        return new Complex(this.real + c.real, this.imag + c.imag);
    }
    void display() {
        System.out.println(real + " + " + imag + "i");
    }
}
public class Main {
    public static void main(String[] args) {
        Complex c1 = new Complex(3, 4);
        Complex c2 = new Complex(2, 5);
        Complex sum = c1.add(c2);    // c2 passed as argument, new object returned
        System.out.print("c1 = "); c1.display();
        System.out.print("c2 = "); c2.display();
        System.out.print("sum = "); sum.display();
    }
}
```

Output:

```
c1 = 3 + 4i
c2 = 2 + 5i
sum = 5 + 9i
```

Explanation

- `c1.add(c2)`: inside `add`, `this` is c1 and `c` is c2. So `this.real + c.real` = 3 + 2 = 5 and `this.imag + c.imag` = 4 + 5 = 9.
- A **new** Complex(5, 9) is created and returned, and stored in `sum`. c1 and c2 are unchanged.

Q10. Access specifiers: comparison table and demonstration

Concept

Access specifiers control **where** a class member can be accessed from. Java has four levels: `private`, default (no keyword), `protected`, `public`.

Specifier	Same class	Same package	Subclass in different package	Different package (non-subclass)
`private`	Yes	No	No	No
default (no keyword)	Yes	Yes	No	No
`protected`	Yes	Yes	Yes (through inheritance)	No
`public`	Yes	Yes	Yes	Yes

Program (all four types, accessed from another class in the same package)

```
class Access {
    private   int a = 1;     // private
              int b = 2;     // default
    protected int c = 3;     // protected
    public    int d = 4;     // public

    void show() {            // inside the same class: all four are accessible
        System.out.println(a + " " + b + " " + c + " " + d);
    }
}
public class Main {         // a different class in the SAME package
    public static void main(String[] args) {
        Access obj = new Access();
        // System.out.println(obj.a);   // COMPILE ERROR: 'a' has private access in Access
        System.out.println(obj.b);      // OK: default is visible inside the same package
        System.out.println(obj.c);      // OK: protected includes package access
        System.out.println(obj.d);      // OK: public is visible everywhere
        obj.show();                     // OK: prints all four
    }
}
```

Output:

```
2
3
4
1 2 3 4
```

Which accesses compile (from class Main, same package)

Access	Compiles?	Reason
`obj.a` (private)	No	private is visible only inside class Access
`obj.b` (default)	Yes	same package
`obj.c` (protected)	Yes	protected includes same-package access
`obj.d` (public)	Yes	public is visible everywhere

From a **different package**, only `public` works for plain access; `protected` works only inside a subclass (through inheritance, not through a parent object reference).

Q11. Inner class and static nested class

Concept

- A **non-static inner class** is a class declared inside another class. Every Inner object is tied to an Outer object and can use **all** members of Outer, even private ones.
- A **static nested class** is declared with `static`. It is **not** tied to an Outer object and can directly use only the **static** members of Outer.

Program

```
class Outer {
    int x = 10;               // instance data member of Outer
    static int y = 20;        // static data member of Outer

    class Inner {             // non-static inner class
        void display() {
            System.out.println("Inner accessing outer x = " + x);
        }
    }
    static class Nested {     // static nested class
        void display() {
            System.out.println("Nested accessing outer static y = " + y);
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Outer o = new Outer();
        Outer.Inner in = o.new Inner();          // an Outer object is required
        in.display();

        Outer.Nested n = new Outer.Nested();     // no Outer object required
        n.display();
    }
}
```

Output:

```
Inner accessing outer x = 10
Nested accessing outer static y = 20
```

Explanation

- `o.new Inner()` creates an Inner object **bound to** the object `o`; that is how `Inner` finds `x`.
- `new Outer.Nested()` works without any Outer object because the nested class is static.
- Inside `Nested` you cannot write `x` directly because there is no Outer object to take it from.

Q12. Anonymous inner classes

Concept

An **anonymous inner class** has **no name**. It is declared and instantiated in one expression: `new TypeName() { ... };`. It either implements an interface or extends a class (including an abstract class), and is used when you need a one-time implementation.

Program

```
interface Greeting {
    void sayHello();
}
abstract class Animal {
    abstract void sound();
}
public class Main {
    public static void main(String[] args) {
        // 1) anonymous class implementing the interface Greeting
        Greeting g = new Greeting() {
            public void sayHello() {
                System.out.println("Hello from anonymous class!");
            }
        };
        g.sayHello();

        // 2) anonymous subclass of the abstract class Animal
        Animal a = new Animal() {
            void sound() {
                System.out.println("Anonymous animal makes a sound");
            }
        };
        a.sound();
    }
}
```

Output:

```
Hello from anonymous class!
Anonymous animal makes a sound
```

Explanation

- `new Greeting() { ... }` does **not** create an object of the interface. It creates an object of a hidden class that implements Greeting. The method inside **must** be `public`.
- `new Animal() { ... }` creates an object of a hidden subclass of Animal that implements the abstract method.
- The reference type (Greeting, Animal) is the parent type; the object has no name you can use.

Q13. The final keyword: variable, method, class

Program (error lines are commented so it compiles)

```
class Base {
    final int MAX = 100;                                        // (a) final variable
    final void show() { System.out.println("Final method"); }    // (b) final method
}
final class Sealed { }                                          // (c) final class

// class Child extends Sealed { }   // ERROR 3: cannot inherit from final class Sealed

class Derived extends Base {
    // void show() { }              // ERROR 2: show() in Derived cannot override final show() in Base
}
public class Main {
    public static void main(String[] args) {
        Base b = new Base();
        // b.MAX = 200;             // ERROR 1: cannot assign a value to final variable MAX
        System.out.println(b.MAX);
        b.show();
    }
}
```

Output:

```
100
Final method
```

Which lines cause compile errors and why

Line	Error	Reason
`b.MAX = 200;`	cannot assign a value to final variable MAX	A final variable can be given a value only once; after that it is a constant.
`void show() { }` in Derived	show() cannot override final method	A final method's body is locked; subclasses may not change it.
`class Child extends Sealed`	cannot inherit from final class	A final class cannot have subclasses (e.g. `String`, `Integer`).

A final variable can also be initialized once later (blank final) inside a constructor. Final **methods can be inherited and called**, only overriding is blocked.

Q14. Abstract class vs interface (table) + Account program

Point	Abstract class	Interface
Keyword	`abstract class`	`interface`
Methods	Abstract and concrete methods both allowed	Abstract by default; default/static/private methods allowed since Java 8/9
Variables	Any kind (instance, static, final, non-final)	Only `public static final` constants
Constructors	Allowed	Not allowed
Inheritance	A class can extend **only one** abstract class	A class can implement **many** interfaces; an interface can extend many interfaces
Access modifiers	Methods can be private, protected, public, default	Methods are public by default
Relationship	IS-A with shared code (closely related classes)	CAN-DO contract (unrelated classes)
Use of keyword	Child uses `extends`	Class uses `implements`

Program: abstract class Account and subclass Savings

```
abstract class Account {
    double balance;
    Account(double balance) {          // abstract classes can have constructors
        this.balance = balance;
    }
    abstract double interest();        // abstract method: no body

    void showBalance() {               // concrete method: shared by all subclasses
        System.out.println("Balance: " + balance);
    }
}
class Savings extends Account {
    Savings(double balance) {
        super(balance);
    }
    double interest() {                // must implement the abstract method
        return balance * 0.04;         // 4% interest
    }
}
public class Main {
    public static void main(String[] args) {
        // Account a = new Account(5000);   // ERROR: Account is abstract; cannot be instantiated
        Account a = new Savings(10000);
        a.showBalance();
        System.out.println("Interest: " + a.interest());
    }
}
```

Output:

```
Balance: 10000.0
Interest: 400.0
```

- An abstract class cannot be instantiated, but a reference of it can hold a subclass object.
- `Savings` must implement `interest()`; otherwise `Savings` also has to be declared abstract.
- Interest = 10000 x 0.04 = 400.0. The balance prints as `10000.0` because it is a `double`.

Q15. Constructor execution order and super(args)

Part 1: both classes have constructors

```
class Parent {
    Parent() { System.out.println("Parent constructor"); }
}
class Child extends Parent {
    Child() { System.out.println("Child constructor"); }
}
public class Main {
    public static void main(String[] args) {
        new Child();
    }
}
```

Output:

```
Parent constructor
Child constructor
```

Order of execution

Step	What happens
1	`new Child()` enters `Child()`.
2	Before the body runs, the compiler-inserted `super();` calls `Parent()`.
3	Parent's body runs: prints **Parent constructor**.
4	Control returns to `Child()`, whose body prints **Child constructor**.

Rule: the superclass constructor always executes first, so the inherited part of the object is ready before the child adds its own part.

Part 2: parent has only a parameterized constructor

```
class Parent {
    Parent(int x) { System.out.println("Parent constructor: " + x); }
}
class Child extends Parent {
    Child() { System.out.println("Child constructor"); }   // COMPILE ERROR
}
```

Error message: `constructor Parent in class Parent cannot be applied to given types; required: int, found: no arguments`.

Why: Java provides a default no-arg constructor **only when a class defines no constructor at all**. Since Parent defines `Parent(int)`, no no-arg constructor exists. `Child()` tries to call the invisible `super();` which finds nothing to call.

Fix: call the parent constructor explicitly with super(args)

```
class Parent {
    Parent(int x) { System.out.println("Parent constructor: " + x); }
}
class Child extends Parent {
    Child() {
        super(10);                      // explicit call, must be the first statement
        System.out.println("Child constructor");
    }
}
public class Main {
    public static void main(String[] args) {
        new Child();
    }
}
```

Output:

```
Parent constructor: 10
Child constructor
```

Two other fixes: add a no-arg constructor `Parent() { }` to Parent, or make `Child` pass a value from its own parameter: `Child(int v) { super(v); }`.

PART C: Theory Questions (3 marks each)

1. Use of the super keyword (two examples)

Definition: `super` is a reference variable that refers to the **immediate parent class** of the current object. It is used inside a subclass.

Use 1: calling the parent class constructor

`super(arguments);` must be the **first statement** of the child constructor. If you do not write it, Java inserts `super();` automatically.

```
class Person {
    String name;
    Person(String name) { this.name = name; }
}
class Student extends Person {
    int roll;
    Student(String name, int roll) {
        super(name);          // calls Person(String)
        this.roll = roll;
    }
    void show() { System.out.println(name + " " + roll); }
}
public class Main {
    public static void main(String[] args) { new Student("Asha", 7).show(); }
}
```

Output:

```
Asha 7
```

Use 2: calling the parent's overridden method

```
class A {
    void show() { System.out.println("Method of A"); }
}
class B extends A {
    void show() {
        super.show();                 // runs A's version
        System.out.println("Method of B");
    }
}
public class Main {
    public static void main(String[] args) { new B().show(); }
}
```

Output:

```
Method of A
Method of B
```

Third use (mention in one line): `super.variable` accesses a parent variable that is hidden by a child variable of the same name.

2. Method overriding and three rules

Definition: When a subclass provides its own implementation of a method that is already defined in its superclass, with the **same name, same parameters and same return type**, it is called **method overriding**. It enables runtime polymorphism.

```
class Animal { void sound() { System.out.println("Animal sound"); } }
class Cat extends Animal {
    @Override
    void sound() { System.out.println("Meow"); }   // overrides Animal.sound()
}
public class Main {
    public static void main(String[] args) {
```

```
        Animal a = new Cat();
        a.sound();
    }
}
```

Output:

```
Meow
```

Rules (write any three)

- The method name and parameter list must be exactly the same; the return type must be the same (or a covariant subtype).
- The access level cannot be more restrictive: a `public` method cannot be overridden as `protected` or default.
- `final`, `static` and `private` methods cannot be overridden.
- The overriding method cannot throw broader (new or higher) checked exceptions than the original.
- There must be inheritance (IS-A relationship) between the two classes.
- Use `@Override` so the compiler checks you really are overriding.

3. Method overloading vs method overriding (three points)

Basis	Method overloading	Method overriding
Meaning	Same method name, different parameters	Subclass redefines a parent's method with the same signature
Where	Within the same class (or inherited)	In subclass and superclass (inheritance needed)
Parameters	Must be different	Must be the same
Polymorphism	Compile-time (static binding)	Runtime (dynamic binding)
Return type	Can differ, but return type alone is not enough	Same or covariant
Access	No restriction	Cannot reduce access
static/final/private	Can be overloaded	Cannot be overridden

4. The final keyword with variables, methods and classes

Used with	Effect	Example
Variable	Becomes a constant; value can be assigned only once	`final int MAX = 10;`
Method	Cannot be overridden by subclasses	`final void show() { }`
Class	Cannot be inherited (no subclass allowed)	`final class A { }`

```
public class Main {
    public static void main(String[] args) {
        final double PI = 3.14;           // constant
        System.out.println(PI);
        // PI = 3.15;                     // ERROR: cannot assign a value to final variable
    }
}
```

Output:

```
3.14
```

Example from the Java library: `String` is a final class, so nobody can extend it.

5. Interface and three differences between a class and an interface

Definition: An interface is a reference type, similar to a class, that contains **abstract method declarations and constants**. It specifies **what** a class must do, and the implementing class decides **how**. It is how Java provides 100% abstraction and multiple inheritance of type.

```
interface Printable { void print(); }
class Doc implements Printable {
    public void print() { System.out.println("Printing document"); }
}
public class Main {
    public static void main(String[] args) {
        Printable p = new Doc();
        p.print();
    }
}
```

Output:

```
Printing document
```

Class	Interface
Can be instantiated with `new`	Cannot be instantiated
Can have constructors	Cannot have constructors
Can have instance variables	Only `public static final` constants
Extended with `extends`; only one parent class	Implemented with `implements`; many interfaces allowed
Methods can be concrete	Methods are abstract by default (default/static allowed from Java 8)

6. The this keyword (two uses with examples)

Definition: `this` is a reference to the **current object** (the object on which the method or constructor is running).

Use 1: to separate instance variables from parameters with the same name

```
class Student {
    int id;
    String name;
    Student(int id, String name) {
        this.id = id;           // this.id = instance variable, id = parameter
        this.name = name;
    }
    void show() { System.out.println(id + " " + name); }
}
public class Main {
    public static void main(String[] args) { new Student(1, "Ravi").show(); }
}
```

Output:

```
1 Ravi
```

Without `this`, `id = id;` would assign the parameter to itself and the instance variable would stay 0.

Use 2: to call another constructor of the same class (constructor chaining)

```
class Box {
    int l, b;
    Box()               { this(1, 1); }      // first statement
    Box(int l, int b)   { this.l = l; this.b = b; }
    void show()         { System.out.println(l + " x " + b); }
}
public class Main {
    public static void main(String[] args) { new Box().show(); }
}
```

Output:

```
1 x 1
```

Other uses (one line): pass the current object as an argument, return the current object (`return this;`), and call a method of the current class.

7. Static variable and static method; why a static method cannot access non-static members

Static variable

Declared with `static`. There is **one copy per class**, shared by all objects, created when the class is loaded. Access with `ClassName.variable`.

Static method

Declared with `static`. It belongs to the class and can be called without creating an object: `ClassName.method()`. It can directly access only static members.

```
class Demo {
    static int s = 5;
    int i = 10;
    static void show() {
        System.out.println(s);        // OK: static member
        // System.out.println(i);     // ERROR: non-static variable i cannot be referenced from a static contex
        Demo d = new Demo();
        System.out.println(d.i);      // OK: through an object reference
    }
}
public class Main {
    public static void main(String[] args) { Demo.show(); }
}
```

Output:

```
5
10
```

Why a static method cannot access non-static members directly

- A static method is loaded with the class and can run **before any object exists**.
- Non-static members belong to **individual objects**. With no object, there is no copy of `i` to read, and no `this` reference inside a static method.
- So the compiler refuses. The method can use non-static members only if it is given an object (like `d.i` above).

8. The finalize() method

Purpose: `finalize()` is a method of the `Object` class that is meant to perform **cleanup** (releasing resources such as files or connections) just before an object is removed from memory.

- **When it is invoked:** by the **garbage collector**, just before it reclaims an object that has no references left. A programmer does not normally call it.

- **Signature:** `protected void finalize() throws Throwable`.

```
public class Main {
    protected void finalize() {
        System.out.println("Object is being garbage collected");
    }
    public static void main(String[] args) {
        Main m = new Main();
        m = null;               // object becomes eligible for garbage collection
        System.gc();            // only a request; the JVM may ignore it
        System.out.println("End of main");
    }
}
```

Output:

```
End of main
Object is being garbage collected    (this line may or may not appear)
```

The finalize message may or may not appear, and its position can vary. That uncertainty is exactly the problem with relying on it.

Why relying on it is discouraged

- There is **no guarantee when, or even whether**, it will run, since garbage collection timing is decided by the JVM.
- It slows down garbage collection and can cause resource leaks. It is **deprecated since Java 9**. Use `try-with-resources` or an explicit `close()` method instead.

9. Static nested class vs inner (non-static) class

Static nested class	Inner (non-static) class
Declared with the `static` keyword	Declared without `static`
Object created without an outer object: `new Outer.Nested()`	Needs an outer object: `outer.new Inner()`
Can directly use only **static** members of the outer class	Can use **all** members of the outer class, including private ones
Can contain static members	Cannot contain static members (traditional Java)
Not tied to an outer instance	Tied to the outer instance that created it

```
class Outer {
    static class Nested { }       // new Outer.Nested()
    class Inner { }               // new Outer().new Inner()
}
```

10. Anonymous inner class and two situations of use

Definition: An anonymous inner class is a class **without a name**, defined and instantiated in a single expression, which extends a class or implements an interface.

```
Runnable r = new Runnable() {          // anonymous class implementing Runnable
    public void run() { System.out.println("Running"); }
};
```

- **Situation 1:** when you need a **one-time implementation** of an interface or abstract class (for example `Runnable`, event listeners, `Comparator`), so a separate named class would be wasteful.
- **Situation 2:** to **override a method of an existing class on the spot** without creating a separate subclass file.

11. Recursion and the role of the base case

Recursion is a technique in which a method **calls itself** to solve a smaller version of the same problem until a simple case is reached.

- **Base case:** the condition where the method returns a direct answer and **stops** calling itself.
- **Recursive case:** the method calls itself with a smaller or simpler input.
- Without a base case the calls never stop and the program fails with `StackOverflowError`.

```
public class Main {
    static int sum(int n) {
        if (n == 0) return 0;          // base case
        return n + sum(n - 1);         // recursive case
    }
    public static void main(String[] args) {
        System.out.println(sum(4));    // 4 + 3 + 2 + 1 + 0
    }
}
```

Output:

```
10
```

12. Constructor overloading and constructor chaining with this()

Constructor overloading: defining more than one constructor in a class, each with a **different parameter list**, so objects can be created in different ways.

Constructor chaining with `this()`: one constructor calls another constructor of the **same class** using `this(...)`. This avoids repeated code. The call must be the **first statement** in the constructor.

```
class Student {
    String name; int age;
    Student()                       { this("Unknown", 18); }   // chains to the next one
    Student(String name)            { this(name, 18); }        // chains to the next one
    Student(String name, int age)   { this.name = name; this.age = age; }
    void show() { System.out.println(name + " " + age); }
}
public class Main {
    public static void main(String[] args) {
        new Student().show();
        new Student("Maya").show();
        new Student("Ravi", 21).show();
    }
}
```

Output:

```
Unknown 18
Maya 18
Ravi 21
```

13. The four access specifiers (one line each)

Specifier	Visibility
`private`	Visible only inside the same class.
default (no keyword)	Visible to all classes in the same package.
`protected`	Visible in the same package and in subclasses of other packages.
`public`	Visible to every class in every package.

Order from most restrictive to least: `private` < default < `protected` < `public`.

14. The instanceof operator and safe type casting

`instanceof` is a binary operator that tests whether an object is an instance of a given class (or subclass) or an interface. It returns `true` or `false`. Syntax: `object instanceof ClassName`.

```
class Animal { }
class Dog extends Animal { void bark() { System.out.println("Bark"); } }
class Cat extends Animal { }
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        if (a instanceof Dog) {            // check first
            Dog d = (Dog) a;               // safe downcast
            d.bark();
        }
        Animal b = new Cat();
        if (b instanceof Dog) {            // false, so the invalid cast is skipped
            Dog d2 = (Dog) b;
            d2.bark();
        } else {
            System.out.println("b is not a Dog, cast avoided");
        }
        // Dog d3 = (Dog) b;   // without the check this throws ClassCastException at runtime
    }
}
```

Output:

```
Bark
b is not a Dog, cast avoided
```

`instanceof` avoids `ClassCastException` by confirming the real type before casting.

15. Abstract class vs interface (three points)

Abstract class	Interface
Can have both abstract and concrete methods	Has abstract methods (default/static allowed since Java 8)
A class can extend only **one** abstract class	A class can implement **many** interfaces
Can have constructors, instance variables, and any access modifiers	No constructors; only `public static final` constants
Declared with `abstract class`; subclasses use `extends`	Declared with `interface`; classes use `implements`

16. Passing objects to and returning objects from methods

- **Passing:** when an object is passed as an argument, a **copy of its reference** is passed (the object itself is not copied). The method works on the **same object**, so changes made inside are visible to the caller.
- **Returning:** a method can return an object by declaring the class name as its return type. The returned value is a reference to the object.

```
class Counter {
    int value;
    Counter(int value) { this.value = value; }

    static void increase(Counter c) {       // object passed in
        c.value = c.value + 10;             // modifies the original object
    }
    static Counter create(int v) {          // object returned
        return new Counter(v);
    }
}
public class Main {
    public static void main(String[] args) {
```

```
        Counter a = Counter.create(5);      // returned object
        Counter.increase(a);                // passed object
        System.out.println(a.value);
    }
}
```

Output:

```
15
```

17. Object vs object reference, and the role of new

Object	Object reference
The actual instance stored in **heap** memory, holding the data	A variable (stored on the stack) that holds the **address** of an object
Created by the `new` operator	Declared like `Student s;`
Exists independently of the variable	Can be null, can be reassigned, many references can point to one object

```
Student s = new Student();

 STACK                 HEAP
+--------+           +--------------------+
|   s    | --------> | Student object     |
| (ref)  |  address  | (fields, data)     |
+--------+           +--------------------+
```

Role of `new`: it (1) allocates memory for the object in the heap, (2) calls the constructor to initialize it, and (3) returns the object's address, which is stored in the reference variable `s`.

18. Multilevel inheritance with a diagram

Definition: In multilevel inheritance a class is derived from another derived class, forming a chain of **grandparent, parent, child**. The last class inherits the members of all classes above it.

```
        +----------------+
        |     Person     |   <- grandparent (name)
        +----------------+
                ^
                |  extends
        +----------------+
        |    Employee    |   <- parent (empId)
        +----------------+
                ^
                |  extends
        +----------------+
        |    Manager     |   <- child (department)
        +----------------+
```

```
class Person   { String name = "Rahul"; }
class Employee extends Person   { int empId = 101; }
class Manager  extends Employee { String dept = "HR"; }
public class Main {
    public static void main(String[] args) {
        Manager m = new Manager();
        System.out.println(m.name + " " + m.empId + " " + m.dept);  // all three levels
    }
}
```

Output:

```
Rahul 101 HR
```

- `Manager` has its own members plus everything from Employee and Person.

- Constructors run **top-down**: Person, then Employee, then Manager.
- Java does **not** allow multiple inheritance with classes (a class cannot extend two classes), but multilevel is allowed.

PART D: Exam Strategy and Revision

Answer-writing format that earns full marks

Question type	What to write
5-mark program	1 line definition of the concept, then the full program with comments, then the **output**, then 2 to 3 lines of explanation.
3-mark theory	Definition, 2 to 3 points (or a table), and a small code example or diagram.
Predict the output	Write the output exactly, line by line. Optionally write a one-line reason beside it.
Differences	Always use a **table** with at least 3 rows.

Common mistakes that lose marks

- Forgetting `public` when implementing interface methods.
- Writing `this()` or `super()` after another statement (it must be the first statement).
- Trying to override a `final`, `static` or `private` method.
- Creating an object of an abstract class or interface with `new` (allowed only as an anonymous class).
- Accessing a non-static member from `static` code without an object.
- Not writing the output of the program.

MCQ revision table

Topic	Remember
static	One shared copy per class; belongs to the class, not to objects
final	Variable = constant; method = cannot override; class = cannot inherit
Access order	private < default < protected < public
Overloading	Compile-time polymorphism; same name, different parameters
Overriding	Runtime polymorphism; same signature in subclass
Interface	Methods public abstract; variables public static final; no constructor
Abstract class	Cannot be instantiated; may have constructors and concrete methods
instanceof	Returns boolean; prevents ClassCastException
finalize()	Called by garbage collector before destroying an object; deprecated since Java 9
this()	Calls another constructor of the same class; must be first statement
super()	Calls the parent constructor; must be first statement; implicit if not written
Inner object	`outer.new Inner()`
Static nested object	`new Outer.Nested()`
Constructor order	Parent first, then child
Multiple inheritance	Not with classes; achieved through interfaces
Default constructor	Provided by Java only if no constructor is written
Method overloading by return type	Not possible

Quick one-line answers for MCQs

- Which keyword prevents inheritance? **final**
- Which method is called by the garbage collector? **finalize()**
- Which operator checks the type of an object? **instanceof**
- Which keyword refers to the current object? **this**
- Which keyword refers to the parent class? **super**
- Parent class of all classes in Java? **Object**
- Can an abstract class have a constructor? **Yes**
- Can an interface have a constructor? **No**
- Can a static method be overridden? **No** (it can be hidden, not overridden)
- Can a class implement multiple interfaces? **Yes**